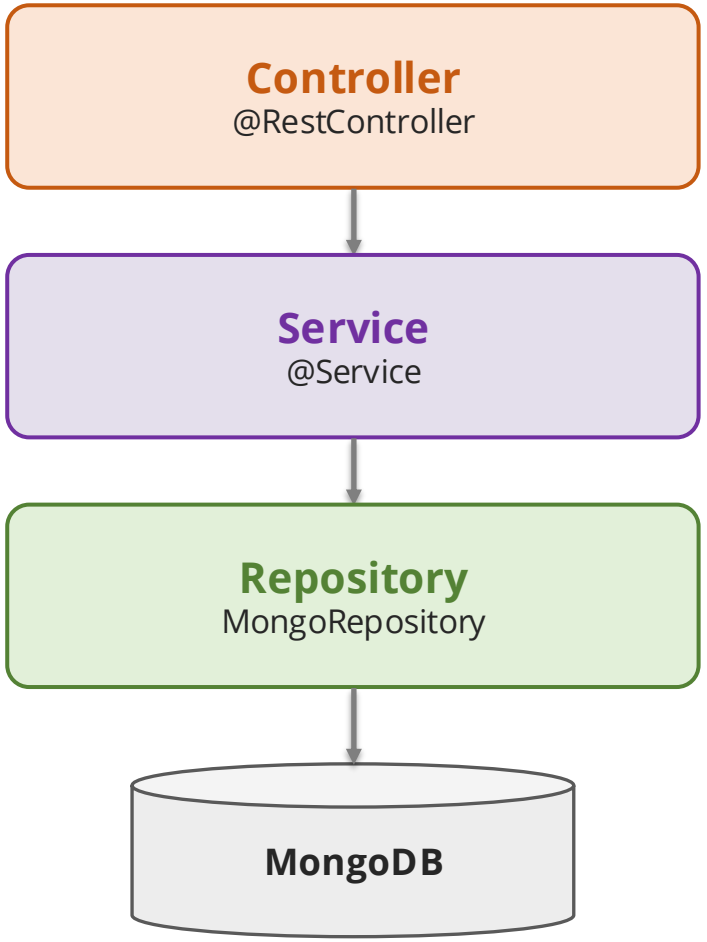
PUC Minas

Desenvolvimento de Aplicações com Spring Boot

Prof. Rommel Vieira Carneiro

Tópicos

- **Introdução ao ecossistema Spring e setup**
  - Spring Framework vs. Spring Boot, Spring Initializr e Starters
- **Criação de APIs REST – a camada Controller**
  - IoC e injeção de dependência, endpoints e captura de dados
- **Persistência com Spring Data MongoDB – a camada Model**
  - Mapeamento de documentos, repositórios e derived queries
- **Boas práticas, validação e tratamento de erros**
  - DTOs, Bean Validation e @RestControllerAdvice
- **Configuração e conexão com o banco de dados**
  - application.properties, perfis e ambientes
- **Testes e publicação**
  - JUnit, Mockito, MockMvc e o JAR executável



Bibliografia da Unidade

WALLS, C. Spring Boot in Action. Manning, 2015.
Bibliografia básica – autoconfiguração, starters e a filosofia do Spring Boot.

XESQUEVIXOS, W. Learning Spring Boot 4. Packt, 2026.
Bibliografia complementar – atualizado para o Spring Boot 4 e o Spring Framework 7.

SPRING. Spring Boot Reference Documentation (docs.spring.io/spring-boot)
Documentação oficial – configuração, starters, testes e empacotamento.

SPRING. Spring Data MongoDB Reference (docs.spring.io/spring-data/mongodb)
Documentação oficial – mapeamento de documentos, repositórios e consultas.

PUC Minas

Spring Boot

Introdução ao Ecossistema Spring e Setup

Prof. Rommel Vieira Carneiro

O ecossistema Spring

O **Spring** é uma família de projetos para aplicações Java corporativas. No centro está o **Spring Framework**; os demais projetos são construídos sobre ele.

Spring Data
MongoDB, JPA, Redis...

Spring Security
autenticação e autorização

Spring Cloud
microsserviços

Spring AI
integração com LLMs

Spring Boot
autoconfiguração · starters · servidor embutido · JAR executável

Spring Framework
IoC/DI · Spring MVC · acesso a dados · transações · testes

Nesta unidade: Spring Boot + Spring MVC (API REST) + Spring Data MongoDB (persistência).

O problema: o Spring “tradicional”

Antes do Spring Boot

- Dezenas de dependências escolhidas e versionadas **à mão** – e compatíveis entre si.
- Configuração extensa em **XML** ou classes Java para cada recurso (MVC, banco, JSON…).
- Aplicação empacotada como **WAR** e implantada em um servidor externo (Tomcat, JBoss).
- Horas de setup antes da primeira linha de regra de negócio.

configuração XML (ilustrativo)

```xml
<!-- dispatcher-servlet.xml (trecho) -->
<beans xmlns="http://www.springframework.org/schema/beans" ...>
  <context:component-scan base-package="br.loja"/>
  <mvc:annotation-driven/>
  <bean id="mongoClient" class="...MongoClientFactoryBean">
    <property name="host" value="localhost"/>
    <property name="port" value="27017"/>
  </bean>
  <bean id="mongoTemplate" class="...MongoTemplate">
    <constructor-arg ref="mongoClient"/>
    <constructor-arg value="loja"/>
  </bean>
  <!-- ... conversores JSON, validação, etc. -->
</beans>
```

O que é o Spring Boot

O **Spring Boot** é uma camada sobre o Spring Framework que aplica **convenção sobre configuração**: decisões sensatas vêm prontas e você só configura o que foge do padrão.



Autoconfiguração

Detecta as bibliotecas no classpath e configura os beans necessários (ex.: MongoDB no classpath → cliente Mongo pronto).



Starters

Dependências agrupadas por funcionalidade, com versões compatíveis garantidas.



Servidor embutido

Tomcat dentro da aplicação: basta executar, sem instalar servidor.



JAR executável

Um único arquivo roda em qualquer máquina com Java: `java -jar app.jar`.

Opinião embutida: a autoconfiguração

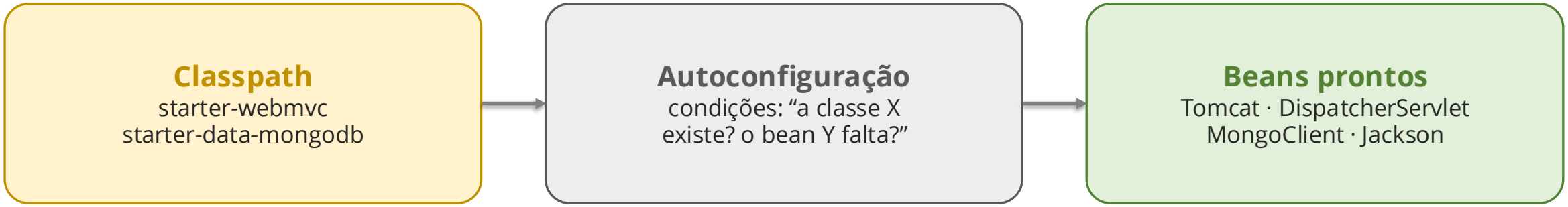
LojaApiApplication.java

```java
@SpringBootApplication
public class LojaApiApplication {

    public static void main(String[] args) {
        SpringApplication.run(LojaApiApplication.class, args);
    }
}
```

`@SpringBootApplication` reúne três anotações:

- `@Configuration` – a classe pode declarar beans
- `@EnableAutoConfiguration` – liga a autoconfiguração
- `@ComponentScan` – procura componentes no pacote e subpacotes



Quase tudo pode ser sobrescrito em application.properties ou declarando o seu próprio bean.

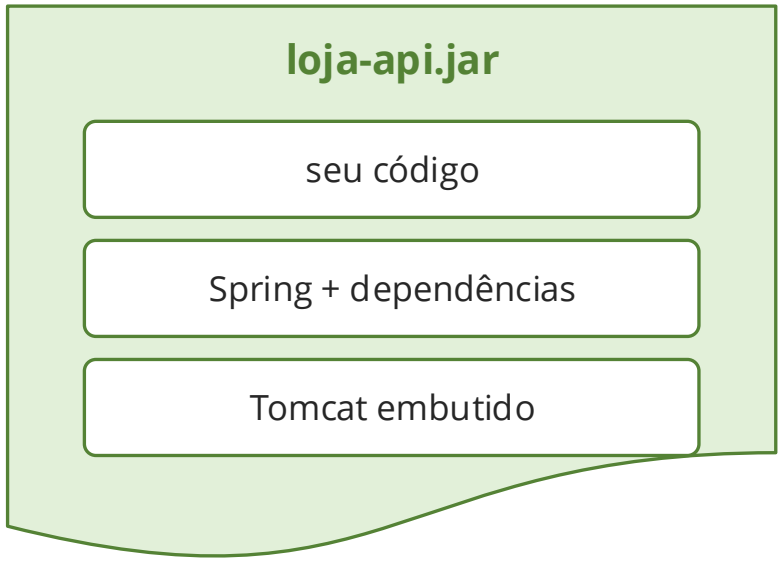
Servidor embutido: WAR vs. JAR

Modelo tradicional (WAR)



O servidor é instalado e configurado à parte; a aplicação é "implantada" dentro dele.

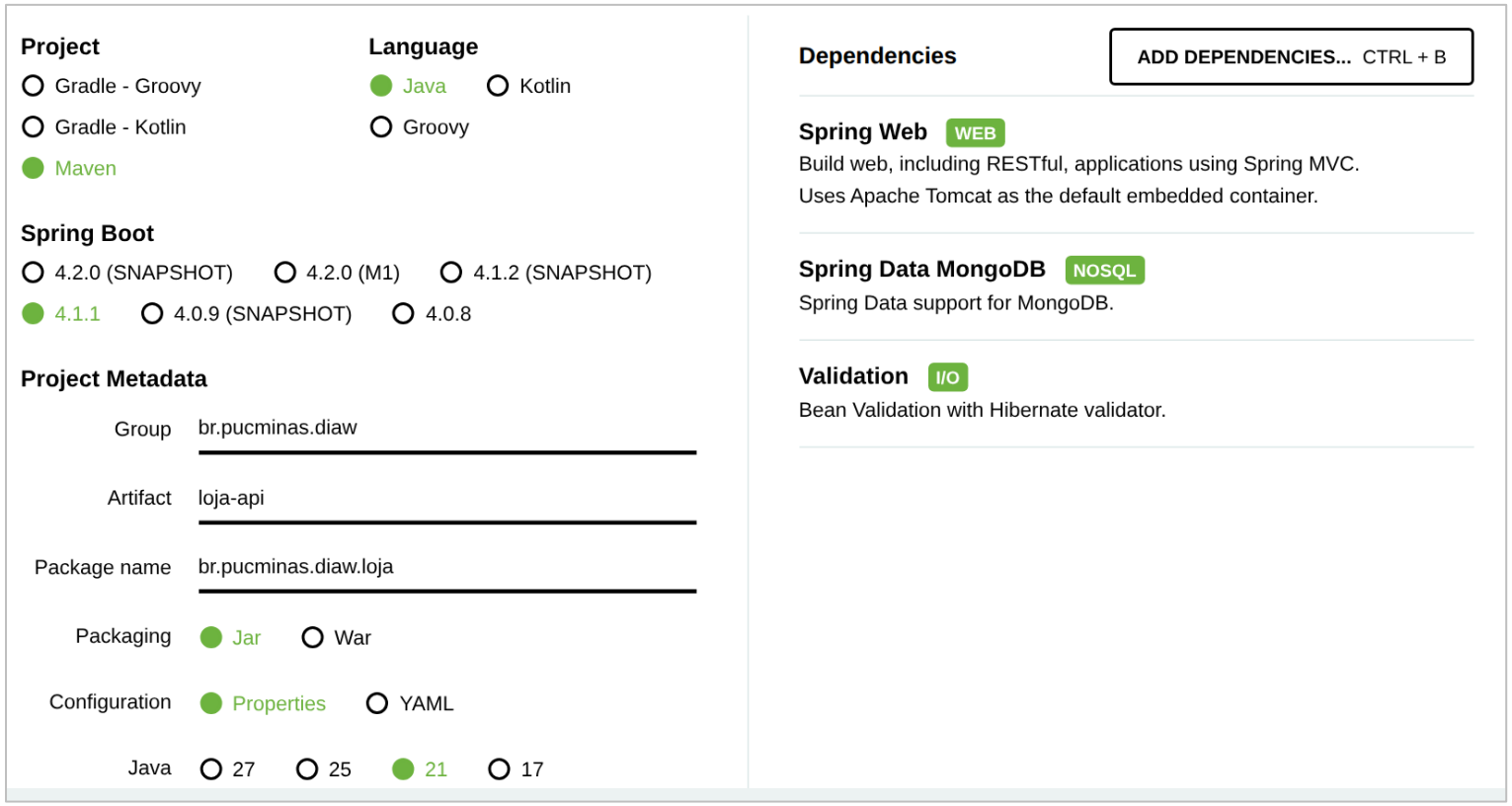
Spring Boot (JAR)



O servidor vai dentro da aplicação: o JAR se executa sozinho.

Recomendação: use JAR. O WAR só se justifica quando a empresa exige um servidor de aplicação já existente.

Spring Initializr



Fonte: https://start.spring.io – captura de tela com a configuração usada na disciplina.

O **Spring Initializr** (start.spring.io) gera o esqueleto do projeto:

- **Project:** Maven – atenção, o padrão do site é Gradle
- **Spring Boot:** última versão estável (sem SNAPSHOT/M)
- **Packaging:** Jar · **Java:** 21
- **Dependências:** Spring Web, Spring Data MongoDB, Validation

No VS Code
Com o **Spring Boot Extension Pack**, o comando "Spring Initializr: Create a Maven Project" oferece as mesmas opções.

Maven ou Gradle? JAR ou WAR?

	Maven	Gradle
Arquivo de build	pom.xml (XML declarativo)	build.gradle(.kts) (Groovy/Kotlin)
Curva de aprendizado	Menor – convenções rígidas	Maior – mais flexível e programável
Desempenho	Bom	Builds incrementais mais rápidos
Uso no mercado	Predominante em projetos Java corporativos	Padrão no Android; comum em projetos grandes

- Na disciplina usamos **Maven**: o `pom.xml` é explícito e fácil de ler para quem está começando.
- O projeto inclui o **Maven Wrapper** (`mvnw`): não é preciso instalar o Maven – o script baixa a versão certa.
- **JAR** é o empacotamento padrão do Spring Boot; **WAR** só para servidores de aplicação legados.

Estrutura do projeto gerado

```
loja-api/
├── mvnw, mvnw.cmd
├── pom.xml
└── src/
    ├── main/
    │   ├── java/br/pucminas/diaw/loja/
    │   │   ├── LojaApiApplication.java
    │   │   ├── controller/
    │   │   ├── service/
    │   │   ├── repository/
    │   │   ├── model/
    │   │   ├── dto/
    │   │   ├── exception/
    │   │   └── config/
    │   └── resources/
    │       └── application.properties
    └── test/java/br/pucminas/diaw/loja/
```

- Build: Maven Wrapper e pom.xml
- Classe principal (main)
- Endpoints REST
- Regras e casos de uso
- Persistência e documentos
- Contratos da API e erros
- Configurações e beans
- Propriedades da aplicação
- Testes automatizados

O arquivo pom.xml

pom.xml (1/2)

```xml
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
    <relativePath/>
</parent>

<groupId>br.pucminas.diaw</groupId>
<artifactId>loja-api</artifactId>
<version>1.0.0</version>

<properties>
    <java.version>21</java.version>
</properties>
```

pom.xml (2/2)

```xml
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-mongodb</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <!-- + starters de teste (scope test) -->
</dependencies>
```

- **parent** herda do Spring Boot as versões compatíveis de todas as bibliotecas – por isso as dependências não informam versão.
- **groupId/artifactId/version** identificam o projeto; **java.version** define o Java; **dependencies** lista os starters.

Starters: dependências por funcionalidade

Um **starter** é uma dependência “guarda-chuva”: declarar um único starter traz todas as bibliotecas necessárias para um recurso, em versões compatíveis.

Starter	O que traz	Uso
spring-boot-starter-webmvc	Spring MVC, Tomcat embutido, Jackson (JSON)	APIs REST e aplicações web
spring-boot-starter-data-mongodb	Driver MongoDB, Spring Data MongoDB	Persistência em documentos
spring-boot-starter-validation	Hibernate Validator (Bean Validation)	Validação de dados
spring-boot-starter-security	Spring Security	Autenticação e autorização
spring-boot-starter-actuator	Endpoints de saúde e métricas	Monitoramento em produção
spring-boot-starter-webmvc-test	JUnit 5, Mockito, AssertJ, MockMvc	Testes automatizados

Spring Boot 4: o antigo spring-boot-starter-web foi renomeado para spring-boot-starter-webmvc.

HANDS ON – Executando o projeto

terminal

```
# Sobe um MongoDB local com Docker (uma única vez)
$ docker run -d --name mongo -p 27017:27017 mongo:8

# Executa a aplicação em modo de desenvolvimento
$ ./mvnw spring-boot:run          # Windows: mvnw.cmd
...
No active profile set, falling back to 1 default profile: "dev"
Tomcat started on port 8080 (http) with context path '/'
Started LojaApiApplication in 3.6 seconds

# Em outro terminal
$ curl localhost:8080/api/produtos
```

1. Gere o projeto no Initializr (ou use o da pasta Fontes)
2. Suba o MongoDB – Docker ou instalação local
3. Execute com `./mvnw spring-boot:run` ou pelo botão Run do VS Code
4. Acesse `localhost:8080/api/produtos` no navegador

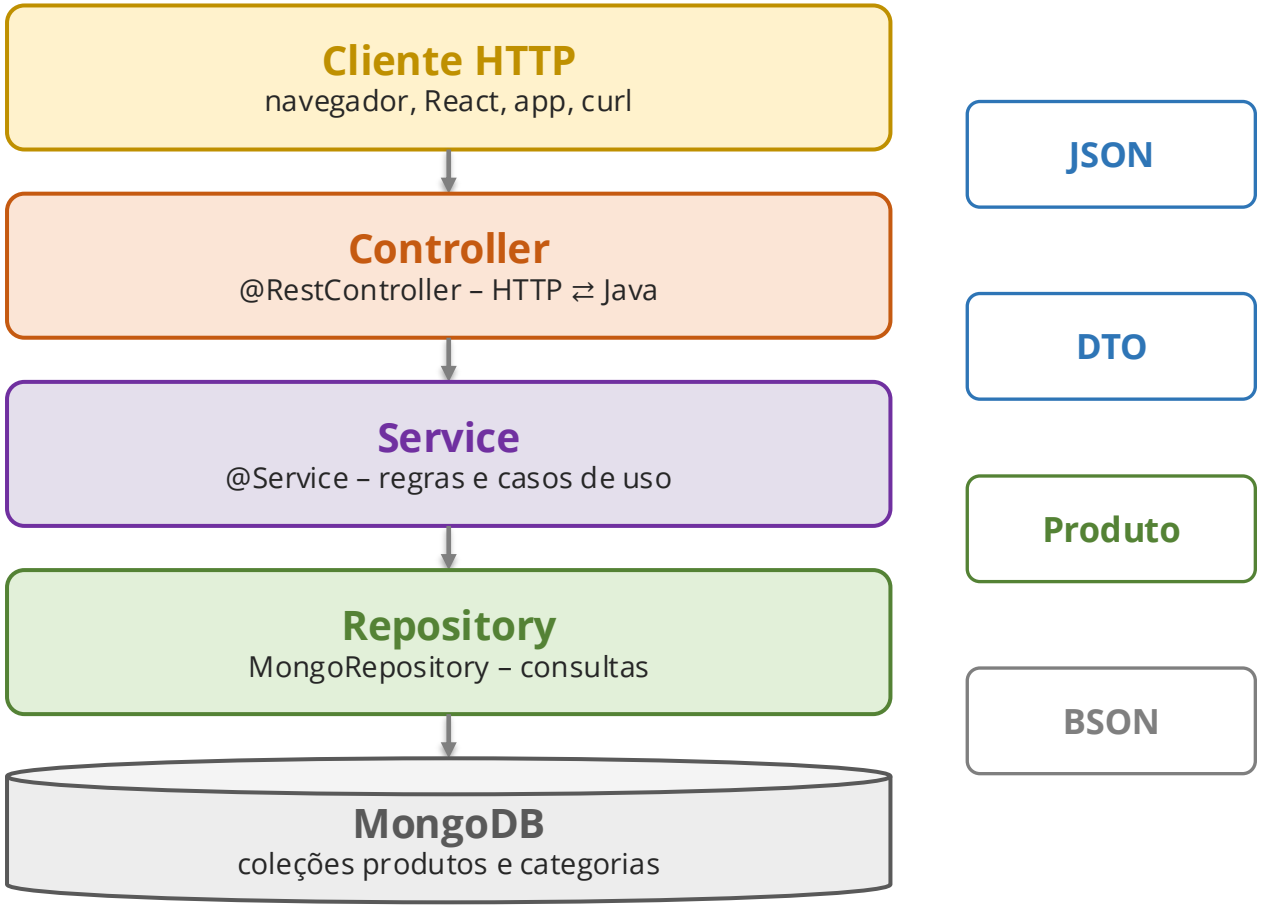
PUC Minas

Spring Boot

Criação de APIs REST (A Camada Controller)

Prof. Rommel Vieira Carneiro

Arquitetura da loja-api



A mesma separação do MVC, agora com **camada de serviço**:

- **Controller** só traduz HTTP: lê a requisição e escolhe o status.
- **Service** concentra os casos de uso (criar produto, validar categoria...).
- **Repository** conversa com o banco.
- **DTOs** são o contrato da API; o **Model** (documento) nunca sai da aplicação.
- A "View" é a representação **JSON** gerada automaticamente pelo Jackson.

Inversão de Controle (IoC)

Na **Inversão de Controle**, os objetos não criam suas dependências: um **contêiner** (o `ApplicationContext` do Spring) cria os objetos – chamados **beans** – e os entrega prontos a quem precisa.

✗ Sem IoC: a classe cria o que usa

```
public class ProdutoService {
  private ProdutoRepository repo =
      new ProdutoRepositoryMongo("localhost", 27017);
  // acoplado à implementação e à configuração;
  // impossível testar sem um MongoDB real
}
```

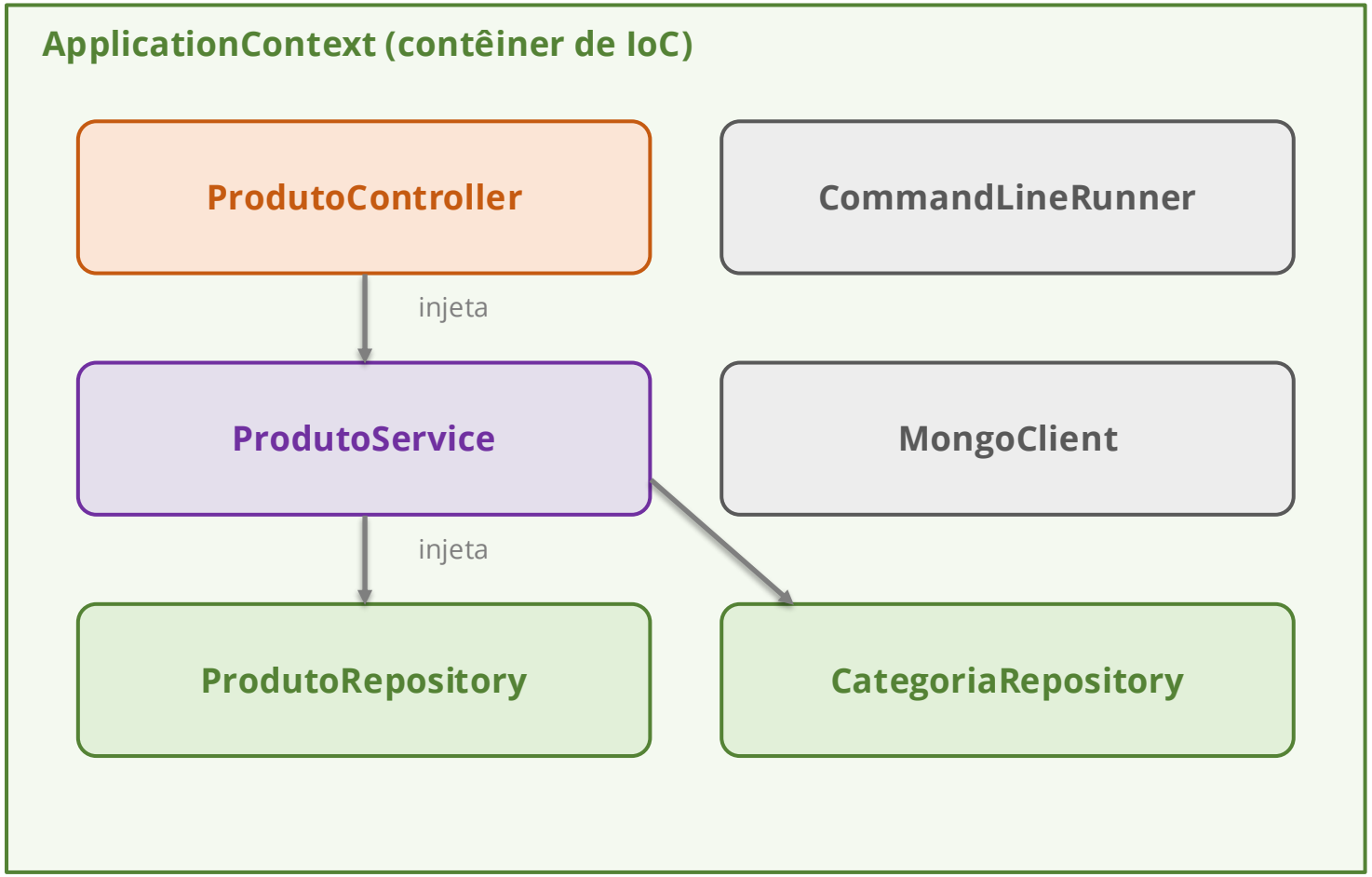
✓ Com IoC: a dependência é injetada

```
@Service
public class ProdutoService {
  private final ProdutoRepository repo;

  public ProdutoService(ProdutoRepository repo) {
    this.repo = repo;   // o Spring entrega pronto
  }
}
```

A **Injeção de Dependência (DI)** é a técnica que implementa a IoC: o contêiner passa as dependências pelo construtor. Resultado: baixo acoplamento e testes com objetos simulados (mocks).

O contêiner Spring em ação



Na inicialização o Spring:

- **varre** os pacotes em busca de classes anotadas (`@Service`, `@RestController`...);
- **cria** um objeto de cada (singleton, por padrão);
- **resolve** as dependências de cada construtor;
- **adiciona** os beans da autoconfiguração (ex.: `MongoClient`).

Estereótipos: marcando os componentes

Anotação	Camada	Significado
`@Component`	Genérica	Classe gerenciada pelo Spring – base das demais
`@Service`	Serviço	Regras de negócio e casos de uso
`@Repository`	Persistência	Acesso a dados; traduz exceções do banco
`@Controller`	Web (MVC)	Controller que devolve Views (HTML)
`@RestController`	Web (API)	`@Controller` + `@ResponseBody`: devolve JSON
`@Configuration`	Configuração	Classe que declara beans com `@Bean`

Interfaces que estendem `MongoRepository` dispensam `@Repository`: o Spring Data gera a implementação e registra o bean automaticamente.

Injeção por construtor vs. @Autowired

Injeção em atributo (evite)

```
@RestController
public class ProdutoController {

  @Autowired
  private ProdutoService service;
}
```

- Dependência escondida; o objeto pode existir "pela metade".
- Testar exige reflexão ou subir o Spring.
- O atributo não pode ser `final`.

Injeção por construtor (recomendada)

```
@RestController
public class ProdutoController {
  private final ProdutoService service;

  public ProdutoController(ProdutoService service) {
    this.service = service;
  }
}
```

- Dependências explícitas e imutáveis (`final`).
- Testável com `new`, passando um mock.
- Com um único construtor, `@Autowired` é **opcional**.

@Bean: registrando objetos manualmente

config/DadosIniciais.java (trecho)

```
@Configuration
@Profile("dev")
public class DadosIniciais {

    @Bean
    CommandLineRunner carregarDados(CategoriaRepository categorias,
                                    ProdutoRepository produtos) {
        return args -> {
            if (produtos.count() > 0) return;
            var perif = categorias.save(new Categoria("Periféricos"));
            // ... demais categorias
            produtos.saveAll(List.of(
                p("Teclado mecânico", "Switch azul", "349.90", 12, perif),
                // ... demais produtos
                p("Pen drive 128 GB", "USB-C", "79.90", 30, armaz)));
        };
    }

    private static Produto p(String nome, String descricao, String preco,
                             int estoque, Categoria categoria) {
        return new Produto(nome, descricao, new BigDecimal(preco),
                estoque, categoria);
    }
}
```

- `@Bean` registra no contêiner o objeto **retornado pelo método**.
- Use quando a classe não é sua (bibliotecas) ou exige montagem especial.
- Os parâmetros do método também são injetados.
- `CommandLineRunner` executa na inicialização: aqui, carrega dados de exemplo.
- `@Profile("dev")`: só existe no perfil de desenvolvimento.

@RestController e @RequestMapping

controller/ProdutoController.java

```
@RestController
@RequestMapping("/api/produtos")
public class ProdutoController {

    private final ProdutoService service;

    public ProdutoController(ProdutoService service) {
        this.service = service;
    }

    @GetMapping
    public PagedModel<ProdutoResponse> listar(
            @RequestParam(required = false) String nome,
            @RequestParam(required = false) String categoria,
            @PageableDefault(size = 10, sort = "nome") Pageable pagina) {
        return new PagedModel<>(service.listar(nome, categoria, pagina));
    }

    @GetMapping("/{id}")
    public ProdutoResponse buscar(@PathVariable String id) {
        return service.buscar(id);
    }
    // ...
}
```

- `@RestController`: o retorno de cada método é convertido em **JSON**.
- `@RequestMapping` define o **prefixo** comum das rotas.
- O Controller só depende do **Service** – não conhece o banco.
- Retornar um objeto ⇒ status **200 OK** por padrão.
- `listar` devolve uma **página** (`PagedModel`) – detalhes adiante.

Mapeamento dos verbos HTTP

Anotação	Método	Rota	Ação	Resposta
`@GetMapping`	GET	/api/produtos	listar (filtros e página)	200 + página
`@GetMapping("/{id}")`	GET	/api/produtos/{id}	buscar	200 ou 404
`@PostMapping`	POST	/api/produtos	criar	201 + Location ou 400
`@PutMapping("/{id}")`	PUT	/api/produtos/{id}	atualizar	200, 400 ou 404
`@DeleteMapping("/{id}")`	DELETE	/api/produtos/{id}	remover	204 ou 404

- Os verbos seguem a semântica do HTTP: GET é **seguro** e **idempotente**; PUT e DELETE são idempotentes; POST não é.
- O mesmo caminho (`/api/produtos/{id}`) atende ações diferentes conforme o método.

Paginação e filtros com Pageable

resposta real da API

```
GET /api/produtos?nome=mouse&categoria=6ab47700…&size=1

{
  "content": [
    {
      "id": "6ab4770067ab644eb4c91387",
      "nome": "Mouse pad XL",
      "descricao": "90 x 40 cm",
      "preco": 89.9,
      "estoque": 25,
      "disponivel": true,
      "categoriaId": "6ab4770067ab644eb4c91380",
      "categoria": "Periféricos"
    }
  ],
  "page": {
    "size": 1,
    "number": 0,
    "totalElements": 2,
    "totalPages": 2
  }
}
```

- `Pageable` recebe `page` (a partir de 0), `size` e `sort` direto da URL.
- `@PageableDefault(size = 10, sort = "nome")` define os valores quando o cliente não informa.
- Derived queries aceitam `Pageable` e devolvem `Page<T>` com o total de elementos e de páginas.
- `PagedModel` padroniza o JSON: `content` + `page`.
- `spring.data.web.pageable.max-page-size=50` impede páginas gigantes.
- Esse formato faz parte do **contrato** que a API Express também segue.

POST, PUT e DELETE com ResponseEntity

controller/ProdutoController.java

```
    @PostMapping
    public ResponseEntity<ProdutoResponse> criar(
            @RequestBody @Valid ProdutoRequest dados) {
        ProdutoResponse criado = service.criar(dados);
        URI local = URI.create("/api/produtos/" + criado.id());
        return ResponseEntity.created(local).body(criado);
    }

    @PutMapping("/{id}")
    public ProdutoResponse atualizar(@PathVariable String id,
            @RequestBody @Valid ProdutoRequest dados) {
        return service.atualizar(id, dados);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> remover(@PathVariable String id) {
        service.remover(id);
        return ResponseEntity.noContent().build();
    }
}
```

- `ResponseEntity` dá controle total sobre **status**, **cabeçalhos** e **corpo**.
- Criação: **201 Created** com o cabeçalho `Location` apontando o novo recurso.
- Exclusão: **204 No Content**, sem corpo.
- `@Valid` dispara a validação do DTO antes de chamar o Service.

Capturando dados da requisição

```
GET  /api/produtos/64f1a2b3?nome=teclado
```

@PathVariable
Parte do caminho

```
@GetMapping("/{id}")
public ProdutoResponse buscar(
    @PathVariable String id)
```

@RequestParam
Query string

```
public PagedModel<…> listar(
    @RequestParam(required = false)
    String nome, ...)
```

@RequestBody
Corpo JSON (POST/PUT)

```
public ResponseEntity<…> criar(
    @RequestBody @Valid
    ProdutoRequest dados)
```

- A conversão de tipos é automática: `String`, números, datas, enums...
- `@RequestBody` usa o **Jackson** para transformar o JSON em um objeto Java (e vice-versa na resposta).

HANDS ON – Testando a API

produtos.http

```
### Listar produtos
GET http://localhost:8080/api/produtos

### Buscar produtos pelo nome
GET http://localhost:8080/api/produtos?nome=teclado

### Listar categorias (copie um id para usar abaixo)
GET http://localhost:8080/api/categorias

### Criar produto
POST http://localhost:8080/api/produtos
Content-Type: application/json

{
  "nome": "Webcam HD",
  "descricao": "1080p com microfone",
  "preco": 219.90,
  "estoque": 7,
  "categoriaId": "COLE_AQUI_O_ID_DA_CATEGORIA"
}

### Criar produto inválido (400)
POST http://localhost:8080/api/produtos
Content-Type: application/json

{ "nome": "TV", "preco": 0, "estoque": -1, "categoriaId": "" }
```

resposta do POST

```
HTTP/1.1 201 Created
Location: /api/produtos/6ab477fd912e0a5bae4d2096
Content-Type: application/json

{
  "id": "6ab477fd912e0a5bae4d2096",
  "nome": "Webcam 4K",
  "descricao": "Autofoco e microfone",
  "preco": 599.9,
  "estoque": 3,
  "disponivel": true,
  "categoriaId": "6ab4770067ab644eb4c91380",
  "categoria": "Periféricos"
}
```

- A extensão **REST Client** do VS Code executa os arquivos `.http` com um clique em “Send Request”.
- Alternativas: curl, Postman, Insomnia, Bruno.

PUC Minas

Spring Boot

Persistência com Spring Data MongoDB (A Camada Model)

Prof. Rommel Vieira Carneiro

Mapeamento objeto-documento (ODM)

Assim como o **ORM** (JPA/Hibernate) mapeia objetos para **tabelas**, o **ODM** do Spring Data MongoDB mapeia objetos Java para **documentos** BSON em coleções.

Conceito	Spring Data JPA (relacional)	Spring Data MongoDB (documentos)
Classe persistente	`@Entity` + `@Table`	`@Document(collection = "produtos")`
Identificador	`@Id` + `@GeneratedValue`	`@Id` (ObjectId gerado automaticamente)
Nome/tipo da coluna ou campo	`@Column`	`@Field`
Índice / unicidade	`@Column(unique = true)`	`@Indexed(unique = true)`
Relacionamento N:1	`@ManyToOne`	`@DocumentReference` ou documento embutido
Repositório	`JpaRepository`	`MongoRepository`

Quem já conhece JPA reconhece o padrão – muda o modelo de dados, não a forma de programar.

O documento Produto

model/Produto.java

```
@Document(collection = "produtos", collation = "pt")
public class Produto {

    @Id
    private String id;

    private String nome;

    private String descricao;

    @Field(targetType = FieldType.DECIMAL128)
    private BigDecimal preco;

    private int estoque;

    @DocumentReference
    private Categoria categoria;

    // construtores, regras de negócio e getters ...
}
```

- `@Document` liga a classe à coleção `produtos`.
- `@Id` do tipo `String` recebe o **ObjectId** gerado pelo MongoDB.
- `BigDecimal` + `DECIMAL128`: valores monetários **sem erro de arredondamento** (nunca use `double` para dinheiro).
- `@DocumentReference` guarda apenas o id da categoria e a carrega ao ler o produto.

Model rico: regras no documento

model/Produto.java (regras)

```
public boolean isDisponivel() {
    return estoque > 0;
}

public void baixarEstoque(int quantidade) {
    if (quantidade <= 0) {
        throw new RegraDeNegocioException("Quantidade inválida");
    }
    if (quantidade > estoque) {
        throw new RegraDeNegocioException("Estoque insuficiente");
    }
    estoque -= quantidade;
}
```

- As regras de negócio ficam no **Model** – como vimos em "Fat Model, Skinny Controller".
- Violações lançam uma exceção de domínio, tratada de forma global (422).
- Sem setters públicos: o estado só muda por métodos que **garantem as regras**.

O Model não conhece HTTP, JSON nem o Controller – pode ser testado com um simples `new Produto(...)`.

Como o produto fica no MongoDB

mongosh › db.produtos.findOne()

```
{
  _id: ObjectId('6ab4770067ab644eb4c91384'),
  nome: 'Teclado mecânico',
  descricao: 'Switch azul',
  preco: Decimal128('349.90'),
  estoque: 12,
  categoria: ObjectId('6ab4770067ab644eb4c91380'),
  _class: 'br.pucminas.diaw.loja.model.Produto'
}
```

- `_id` é o **ObjectId** gerado pelo banco.
- `preco` gravado como **Decimal128**.
- `categoria` guarda só a **referência** (id) – como uma chave estrangeira.
- `_class` é adicionado pelo Spring Data para reconstruir o tipo Java.

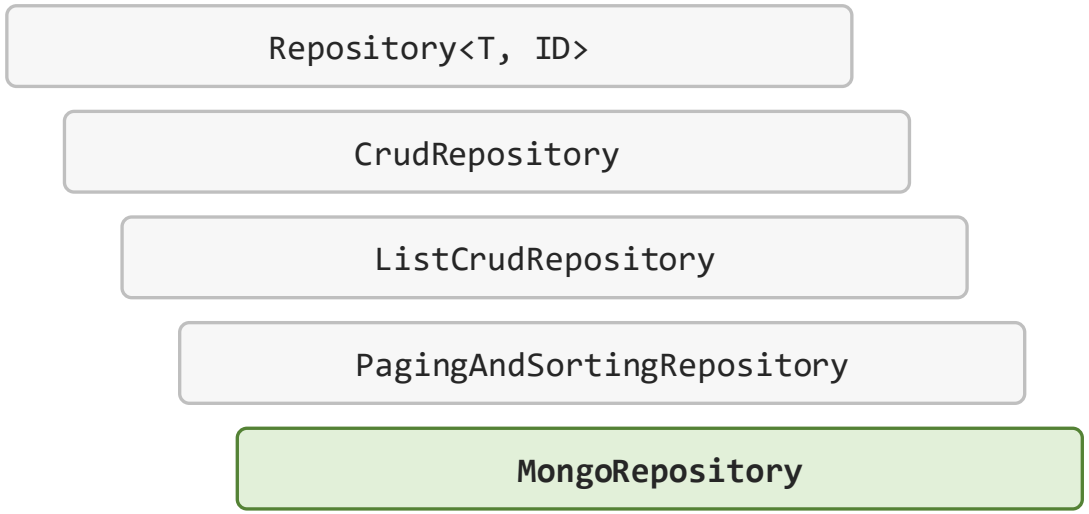
Referenciar ou embutir? Essa decisão de modelagem será aprofundada na unidade de MongoDB.

Spring Data Repositories

repository/CategoriaRepository.java

```
public interface CategoriaRepository
        extends MongoRepository<Categoria, String> {

    Optional<Categoria> findByNomeIgnoreCase(String nome);
}
```



Método herdado	Ação
`save(entidade)`	Insere ou atualiza
`findById(id)`	Busca → `Optional`
`findAll()`	Lista todos
`deleteById(id)`	Remove
`count()`	Conta documentos
`existsById(id)`	Verifica existência
`findAll(Pageable)`	Paginação e ordenação

Nenhuma linha de implementação: o Spring Data gera a classe em tempo de execução.

Derived Queries: consultas pelo nome

repository/ProdutoRepository.java

```
public interface ProdutoRepository
        extends MongoRepository<Produto, String> {

    Page<Produto> findByNomeContainingIgnoreCase(String nome,
                                                 Pageable pageable);

    Page<Produto> findByCategoria(Categoria categoria, Pageable pageable);

    Page<Produto> findByNomeContainingIgnoreCaseAndCategoria(
            String nome, Categoria categoria, Pageable pageable);

    List<Produto> findByPrecoBetweenOrderByPrecoAsc(BigDecimal min,
                                                    BigDecimal max);
}
```

- Basta **declarar** o método: o Spring Data interpreta o nome e gera a consulta.
- Com `Pageable` no último parâmetro, a consulta é **paginada** e devolve `Page<T>`.
- `And`, `Or`, `Between`, `OrderBy`... combinam critérios.
- Erros de nome são detectados na **inicialização** da aplicação.

Derived Queries: anatomia do nome

O nome do método é dividido em partes com significado:



ação · critério · propriedade · operador · modificador

- `findByNomeContainingIgnoreCase("teclado")` gera `{ nome: { $regex: "teclado", $options: "i" } }`.
- Nomes muito longos? Use `@Query` com a consulta em JSON.

Palavra-chave	Consulta MongoDB
`Containing`	`$regex`
`Between`	`$gt` e `$lt`
`GreaterThan`	`$gt`
`OrderBy…Asc`	ordenação

A camada de serviço (1/3)

service/ProdutoService.java

```
@Service
public class ProdutoService {

    private final ProdutoRepository produtos;
    private final CategoriaRepository categorias;

    public ProdutoService(ProdutoRepository produtos,
                          CategoriaRepository categorias) {
        this.produtos = produtos;
        this.categorias = categorias;
    }

    public ProdutoResponse buscar(String id) {
        return ProdutoResponse.de(buscarProduto(id));
    }
    // ...
}
```

- `@Service` registra a classe como bean.
- Recebe os dois repositórios pelo construtor.
- Converte cada documento em **DTO** de resposta.

A camada de serviço (2/3)

service/ProdutoService.java

```
// ...
public Page<ProdutoResponse> listar(String nome, String categoriaId,
                                    Pageable pagina) {
    boolean porNome = nome != null && !nome.isBlank();
    Page<Produto> resultado;
    if (categoriaId != null && !categoriaId.isBlank()) {
        Categoria categoria = buscarCategoria(categoriaId);
        resultado = porNome
                ? produtos.findByNomeContainingIgnoreCaseAndCategoria(
                        nome, categoria, pagina)
                : produtos.findByCategoria(categoria, pagina);
    } else {
        resultado = porNome
                ? produtos.findByNomeContainingIgnoreCase(nome, pagina)
                : produtos.findAll(pagina);
    }
    return resultado.map(ProdutoResponse::de);
}
// ...
```

- Escolhe a derived query conforme os filtros recebidos.
- Repassa o `Pageable` vindo do Controller.
- `Page.map` converte cada documento em DTO sem perder os dados da página.
- Categoria inexistente no filtro → 404.

A camada de serviço (3/3)

service/ProdutoService.java

```
    // ...
    public ProdutoResponse criar(ProdutoRequest dados) {
        Categoria categoria = buscarCategoria(dados.categoriaId());
        Produto produto = new Produto(dados.nome(), dados.descricao(),
                dados.preco(), dados.estoque(), categoria);
        return ProdutoResponse.de(produtos.save(produto));
    }

    public ProdutoResponse atualizar(String id, ProdutoRequest dados) {
        Produto produto = buscarProduto(id);
        Categoria categoria = buscarCategoria(dados.categoriaId());
        produto.atualizar(dados.nome(), dados.descricao(),
                dados.preco(), dados.estoque(), categoria);
        return ProdutoResponse.de(produtos.save(produto));
    }

    public void remover(String id) {
        produtos.delete(buscarProduto(id));
    }

    // buscarProduto(id) e buscarCategoria(id): findById(...).orElseThrow(...)
}
```

- Orquestra o caso de uso: valida a categoria, cria ou altera o produto e salva.
- `save` insere quando não há id e atualiza quando há.
- Recurso inexistente → exceção de domínio, convertida em 404.
- Nenhuma referência a HTTP.

PUC Minas

Spring Boot

Boas Práticas, Validação e Tratamento de Erros

Prof. Rommel Vieira Carneiro

DTOs: por que não expor o documento?

Mass assignment

O cliente poderia enviar campos que não deveria alterar (id, estoque reservado...).

Vazamento de dados

Campos internos (custos, senhas, `_class`) iriam para a resposta.

Contrato instável

Mudar o banco mudaria a API e quebraria os clientes.

Formato adequado

A resposta pode ter campos calculados (`disponivel`) e dados achatados (`categoria`).



O DTO de entrada chega ao Service, vira documento e volta como DTO de saída.

DTO de entrada com validação

dto/ProdutoRequest.java

```
public record ProdutoRequest(
        @NotBlank(message = "Nome é obrigatório")
        @Size(min = 3, max = 100, message = "Nome: de 3 a 100 caracteres")
        String nome,

        @Size(max = 500, message = "Descrição: até 500 caracteres")
        String descricao,

        @NotNull(message = "Preço é obrigatório")
        @Positive(message = "Preço deve ser maior que zero")
        BigDecimal preco,

        @NotNull(message = "Estoque é obrigatório")
        @PositiveOrZero(message = "Estoque não pode ser negativo")
        Integer estoque,

        @NotBlank(message = "Categoria é obrigatória")
        String categoriaId
) { }
```

- Um **record** (Java 16+) é imutável e gera construtor, acessores, `equals` e `toString`.
- As anotações declaram as **regras de entrada** da API.
- `message` define o texto devolvido ao cliente.

DTO de saída

dto/ProdutoResponse.java

```
public record ProdutoResponse(
        String id,
        String nome,
        String descricao,
        BigDecimal preco,
        int estoque,
        boolean disponivel,
        String categoriaId,
        String categoria
) {
    public static ProdutoResponse de(Produto p) {
        return new ProdutoResponse(
                p.getId(), p.getNome(), p.getDescricao(),
                p.getPreco(), p.getEstoque(), p.isDisponivel(),
                p.getCategoria().getId(), p.getCategoria().getNome());
    }
}
```

```
{ "id": "6ab4…", "nome": "Teclado mecânico", "descricao": "Switch azul", "preco": 349.90,
  "estoque": 12, "disponivel": true, "categoriaId": "6ab4…", "categoria": "Periféricos" }
```

- Expõe apenas o que o cliente precisa.
- `disponivel` é **calculado** pela regra do Model.
- `categoriaId` + `categoria` (nome): prontos para formulários e listagens.
- `de(Produto)` centraliza a conversão.

Validação com Bean Validation

Anotação	Regra
`@NotNull`	Não pode ser nulo
`@NotBlank`	Texto não nulo e não vazio
`@Size(min, max)`	Tamanho de texto ou lista
`@Min` / `@Max`	Limites numéricos
`@Positive` / `@PositiveOrZero`	Número > 0 / ≥ 0
`@Email`	Formato de e-mail
`@Pattern(regexp)`	Expressão regular
`@Past` / `@Future`	Datas no passado / futuro

- Requer o **spring-boot-starter-validation**.
- As regras ficam no **DTO**, junto ao contrato da API.
- `@Valid` no parâmetro do Controller dispara a validação **antes** do método executar.
- Falhou? O Spring lança `MethodArgumentNotValidException` → **400 Bad Request**.
- A validação do DTO não substitui as **regras de negócio** do Model.

Sem tratamento: erros confusos e inseguros

Sem um tratamento global, uma exceção não prevista vira uma resposta **500** genérica – e, mal configurada, pode expor detalhes internos:

ilustrativo

```
HTTP/1.1 500 Internal Server Error
{
  "timestamp": "2026-09-23T14:02:11.412Z",
  "status": 500,
  "error": "Internal Server Error",
  "trace": "br.pucminas...RecursoNaoEncontrado
    Exception: Produto x não encontrado
    at br.pucminas.diaw.loja.service...
    at org.springframework.web...",
  "path": "/api/produtos/x"
}
```

- Status errado: “não encontrado” deveria ser **404**, não 500.
- O **stack trace** revela pacotes, bibliotecas e versões – informação útil para um atacante.
- Cada erro tem um formato diferente: o front-end não sabe o que exibir.

Solução: centralizar o tratamento em um @RestControllerAdvice com respostas no padrão RFC 9457 (Problem Details).

Tratamento global de exceções (1/2)

exception/GlobalExceptionHandler.java

```
@RestControllerAdvice
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

    @ExceptionHandler(RecursoNaoEncontradoException.class)
    public ProblemDetail naoEncontrado(RecursoNaoEncontradoException ex) {
        return ProblemDetail.forStatusAndDetail(
                HttpStatus.NOT_FOUND, ex.getMessage());
    }

    @ExceptionHandler(RegraDeNegocioException.class)
    public ProblemDetail regraDeNegocio(RegraDeNegocioException ex) {
        return ProblemDetail.forStatusAndDetail(
                HttpStatus.UNPROCESSABLE_CONTENT, ex.getMessage());
    }
    // ...
}
```

- `@RestControllerAdvice` intercepta exceções de **todos** os controllers.
- Cada `@ExceptionHandler` associa uma exceção a um status HTTP.
- `ProblemDetail` gera o JSON padrão da **RFC 9457**.
- Os Services lançam exceções de domínio; o HTTP fica só aqui.

Tratamento global de exceções (2/2)

exception/GlobalExceptionHandler.java

```
    // ...
    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
            MethodArgumentNotValidException ex, HttpHeaders headers,
            HttpStatusCode status, WebRequest request) {
        Map<String, String> erros = ex.getFieldErrors().stream()
                .collect(Collectors.toMap(FieldError::getField,
                        FieldError::getDefaultMessage, (a, b) -> a));
        ProblemDetail problema = ex.getBody();
        problema.setDetail("Dados inválidos");
        problema.setProperty("erros", erros);
        return ResponseEntity.badRequest().body(problema);
    }

    @Override
    protected ResponseEntity<Object> handleHttpMessageNotReadable(
            HttpMessageNotReadableException ex, HttpHeaders headers,
            HttpStatusCode status, WebRequest request) {
        return ResponseEntity.badRequest().body(ProblemDetail
                .forStatusAndDetail(HttpStatus.BAD_REQUEST, "JSON malformado"));
    }

    @ExceptionHandler(Exception.class)
    public ProblemDetail erroInesperado(Exception ex) {
        log.error("Erro inesperado", ex);
        return ProblemDetail.forStatusAndDetail(
                HttpStatus.INTERNAL_SERVER_ERROR,
                "Erro interno. Tente novamente mais tarde.");
    }
}
```

- Herdar de `ResponseEntityExceptionHandler` mantém os erros do próprio Spring (JSON malformado, 405, 415) no mesmo formato.
- Sobrescrevemos só a validação, para listar os campos com erro.
- O handler genérico registra o erro no **log** e devolve mensagem neutra – sem stack trace.

Respostas de erro padronizadas

POST inválido → 400

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "detail": "Dados inválidos",
  "instance": "/api/produtos",
  "status": 400,
  "title": "Bad Request",
  "erros": {
    "preco": "Preço deve ser maior que zero",
    "estoque": "Estoque não pode ser negativo",
    "nome": "Nome: de 3 a 100 caracteres",
    "categoriaId": "Categoria é obrigatória"
  }
}
```

GET inexistente → 404

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{
  "detail": "Produto abc123 não encontrado",
  "instance": "/api/produtos/abc123",
  "status": 404,
  "title": "Not Found"
}
```

- Mesmo formato para todos os erros: `type`, `title`, `status`, `detail`, `instance`.
- Campos extras (`erros`) detalham cada problema de validação.
- Content-Type: `application/problem+json`.
- JSON malformado → 400 · método errado → 405 · formato errado → 415.

PUC Minas

Spring Boot

Configuração e Conexão com Banco de Dados

Prof. Rommel Vieira Carneiro

application.properties

application.properties

```
spring.application.name=loja-api

# Servidor
server.port=8080

# MongoDB (valor padrão para desenvolvimento local)
spring.mongodb.uri=${MONGODB_URI:mongodb://localhost:27017/loja}
spring.data.mongodb.auto-index-creation=true

# Paginação: tamanho máximo de página aceito pela API
spring.data.web.pageable.max-page-size=50

# Origens autorizadas a chamar a API pelo navegador (CORS)
app.cors.origens=${CORS_ORIGENS:http://localhost:3001}

# Perfil ativo por padrão
spring.profiles.default=dev
```

equivalente em application.yml (trecho)

```
spring:
  application:
    name: loja-api
  mongodb:
    uri: ${MONGODB_URI:mongodb://localhost:27017/loja}
```

- Centraliza a configuração: porta, conexão com o banco, logs...
- `${MONGODB_URI:valor}` lê uma **variável de ambiente** e usa o valor após `:` como padrão.
- `.properties` ou `.yml`: mesmo efeito, sintaxes diferentes.
- Spring Boot 4: as propriedades de conexão passaram de `spring.data.mongodb.*` para `spring.mongodb.*`.

CORS: liberando o acesso do front-end

config/CorsConfig.java

```java
@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Value("${app.cors.origens}")
    private String[] origens;

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
                .allowedOrigins(origens)
                .allowedMethods("GET", "POST", "PUT", "DELETE")
                .exposedHeaders("Location");
    }
}
```

application.properties

```
app.cors.origens=${CORS_ORIGENS:http://localhost:3001}
```

- O navegador bloqueia chamadas JavaScript para **outra origem** (domínio, protocolo ou porta) – a *Same-Origin Policy*.
- O front-end Next.js (porta 3001) só consegue chamar a API (porta 8080) se ela autorizar a origem via **CORS**.
- `@Value` injeta a lista de origens a partir das propriedades.
- Nunca use `*` em produção: autorize só as origens conhecidas.

Perfis: um código, vários ambientes

perfil dev	perfil test	perfil prod
Desenvolvimento MongoDB local (Docker) dados de exemplo carregados	**Testes** banco loja-test isolado (ou Testcontainers)	**Produção** MongoDB Atlas credenciais via variável de ambiente

application-prod.properties

```
# Produção: a URI (com usuário e senha) vem SEMPRE de variável de ambiente
spring.mongodb.uri=${MONGODB_URI}
server.port=${PORT:8080}
```

```
$ export MONGODB_URI="mongodb+srv://..."
$ java -jar loja-api.jar \
    --spring.profiles.active=prod
```

Nunca coloque senhas no código ou no Git
Credenciais vêm de variáveis de ambiente ou de um cofre de segredos. Um `.env` com senhas deve estar no `.gitignore`.

Banco de desenvolvimento vs. produção

	Desenvolvimento	Testes	Produção
Onde roda	Docker ou instalação local	Banco isolado / Testcontainers	MongoDB Atlas (nuvem)
Conexão	mongodb://localhost:27017/loja	.../loja-test	mongodb+srv://... (TLS)
Dados	Exemplo (CommandLineRunner)	Criados e apagados a cada teste	Reais, com backup
Credenciais	Não exige	Não exige	Usuário/senha via variável

E o H2?
O **H2** é um banco **relacional** em memória, muito usado com JPA para testes rápidos. Com MongoDB, o equivalente é subir uma instância descartável: um contêiner Docker ou o **Testcontainers**, que cria e destrói o banco automaticamente durante os testes.

PUC Minas

Spring Boot

Testes e Publicação (Produção)

Prof. Rommel Vieira Carneiro

Estratégia de testes



@SpringBootTest
Velocidade: mais lento

ProdutoControllerTest · ProdutoRepositoryTest
Velocidade: segundos

ProdutoTest · ProdutoServiceTest
Velocidade: milissegundos

Muitos testes rápidos na base, poucos testes lentos no topo. Todos vêm com o spring-boot-starter-*-test.

Teste unitário do Model

ProdutoTest.java

```
class ProdutoTest {

    private final Produto teclado = new Produto("Teclado", "ABNT2",
            new BigDecimal("349.90"), 2, new Categoria("Periféricos"));

    @Test
    void deveBaixarEstoque() {
        teclado.baixarEstoque(2);
        assertThat(teclado.getEstoque()).isZero();
        assertThat(teclado.isDisponivel()).isFalse();
    }

    @Test
    void naoDeveBaixarMaisQueOEstoque() {
        assertThatThrownBy(() -> teclado.baixarEstoque(3))
                .isInstanceOf(RegraDeNegocioException.class)
                .hasMessageContaining("Estoque insuficiente");
    }
}
```

- **JUnit 5**: `@Test` marca cada caso de teste.
- **AssertJ**: asserções legíveis (`assertThat…isZero()`).
- Nenhum Spring, nenhum banco: executa em milissegundos.
- Só é possível porque a regra está no **Model**.

Teste do Service com Mockito

ProdutoServiceTest.java

```
@ExtendWith(MockitoExtension.class)
class ProdutoServiceTest {

    @Mock ProdutoRepository produtos;
    @Mock CategoriaRepository categorias;
    @InjectMocks ProdutoService service;

    private final ProdutoRequest dados = new ProdutoRequest(
            "Webcam HD", "1080p", new BigDecimal("219.90"), 7, "cat-1");

    @Test
    void deveCriarProdutoComCategoriaExistente() {
        when(categorias.findById("cat-1"))
                .thenReturn(Optional.of(new Categoria("Periféricos")));
        when(produtos.save(any(Produto.class)))
                .thenAnswer(inv -> inv.getArgument(0));

        var criado = service.criar(dados);

        assertThat(criado.nome()).isEqualTo("Webcam HD");
        assertThat(criado.categoria()).isEqualTo("Periféricos");
        assertThat(criado.disponivel()).isTrue();
    }
    // + teste: categoria inexistente → exceção e save() nunca chamado
}
```

- `@Mock` cria repositórios **simulados**.
- `@InjectMocks` cria o Service injetando os mocks pelo construtor.
- `when(...).thenReturn(...)` programa o comportamento.
- `verify` confere que o produto **não** foi salvo.

Teste do Controller com MockMvc

ProdutoControllerTest.java

```
@WebMvcTest(ProdutoController.class)
class ProdutoControllerTest {

    @Autowired MockMvc mvc;
    @MockitoBean ProdutoService service;

    @Test
    void deveRetornarProdutoPorId() throws Exception {
        var teclado = new ProdutoResponse("p1", "Teclado", "ABNT2",
                new BigDecimal("349.90"), 12, true, "c1", "Periféricos");
        when(service.buscar("p1")).thenReturn(teclado);

        mvc.perform(get("/api/produtos/p1"))
           .andExpect(status().isOk())
           .andExpect(jsonPath("$.nome").value("Teclado"))
           .andExpect(jsonPath("$.disponivel").value(true));
    }

    // + testes: 404 (produto inexistente) e 400 (JSON inválido)
}
```

- `@WebMvcTest` sobe **só a camada web** – sem banco.
- `MockMvc` simula requisições HTTP sem abrir porta.
- `@MockitoBean` substitui o Service por um mock no contexto.
- `jsonPath` verifica campos do JSON de resposta.

Teste do repositório e de integração

ProdutoRepositoryTest.java

```java
@DataMongoTest
class ProdutoRepositoryTest {

    @Autowired ProdutoRepository produtos;
    @Autowired CategoriaRepository categorias;

    private Categoria perifericos;

    @BeforeEach
    void preparar() {
        produtos.deleteAll();
        categorias.deleteAll();
        perifericos = categorias.save(new Categoria("Periféricos"));
        produtos.save(new Produto("Teclado mecânico", null,
                new BigDecimal("349.90"), 12, perifericos));
        produtos.save(new Produto("Mouse sem fio", null,
                new BigDecimal("129.90"), 0, perifericos));
    }

    @Test
    void deveBuscarPorParteDoNome() {
        var pagina = produtos.findByNomeContainingIgnoreCase("TECLADO",
                PageRequest.of(0, 10));
        assertThat(pagina.getContent())
                .extracting(Produto::getNome)
                .containsExactly("Teclado mecânico");
    }

    // + testes: por categoria e por faixa de preço
}
```

- `@DataMongoTest` sobe só a camada de persistência.
- Usa um **banco real** (loja-test) – valida as derived queries de verdade.
- `@SpringBootTest` sobe a aplicação **inteira**: use com moderação.
- Evolução: **Testcontainers** cria um MongoDB descartável via Docker.

Executando os testes

terminal

```
$ ./mvnw test

[INFO] Tests run: 2, Failures: 0, Errors: 0, 0.102 s -- in model.ProdutoTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, 2.582 s -- in repository.ProdutoRepositoryTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, 0.328 s -- in service.ProdutoServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, 1.356 s -- in controller.ProdutoControllerTest
[INFO] Tests run: 10, Failures: 0, Errors: 0
[INFO] BUILD SUCCESS
[INFO] Total time:  7.529 s
```

- `./mvnw test` executa todos os testes; o build **falha** se algum teste falhar.
- No VS Code, o painel **Testing** executa e depura cada teste individualmente.

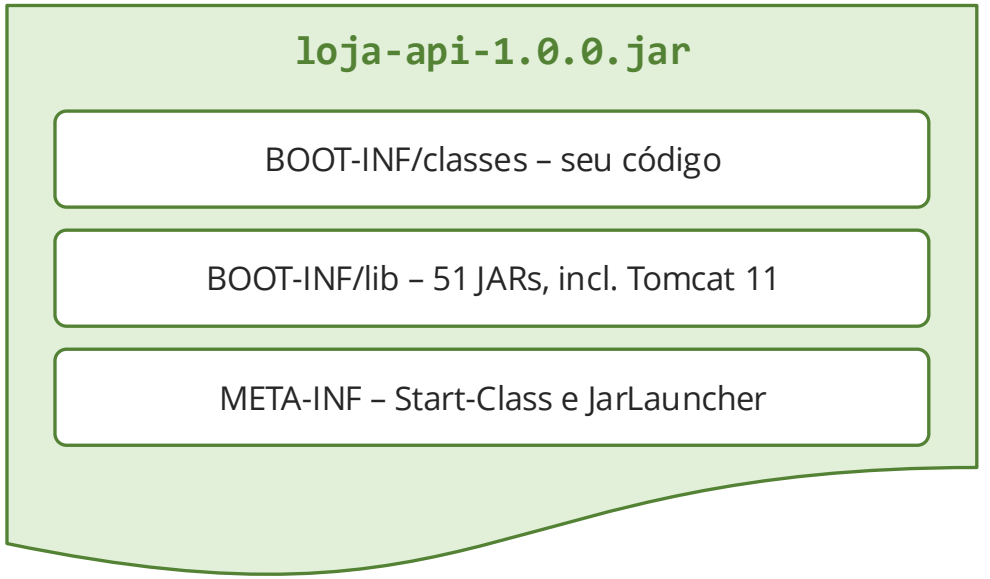
Gerando o executável

terminal

```
# Gera o JAR (executa os testes antes)
$ ./mvnw clean package
[INFO] Tests run: 10, Failures: 0, Errors: 0
[INFO] Building jar: target/loja-api-1.0.0.jar
[INFO] BUILD SUCCESS

# Executa em produção
$ export MONGODB_URI="mongodb+srv://usuario:senha@..."
$ java -jar target/loja-api-1.0.0.jar \
       --spring.profiles.active=prod

INFO The following 1 profile is active: "prod"
INFO Tomcat started on port 8080 (http)
INFO Started LojaApiApplication in 3.683 seconds
```



- **fat JAR**: autossuficiente, basta ter Java instalado.
- O mesmo arquivo roda em qualquer ambiente; o **perfil** e as **variáveis** mudam o comportamento.

Desafios

1 CRUD de categorias

Complete o `CategoriaController` com POST, PUT e DELETE, usando DTOs, validação e um Service.

2 Filtro por faixa de preço

Exponha `GET /api/produtos?min=100&max=500` usando a derived query `findByPrecoBetween...`.

3 Baixa de estoque

Crie `POST /api/produtos/{id}/baixa` que chama `baixarEstoque` e retorna 422 quando faltar estoque.

4 Produtos esgotados

Crie `GET /api/produtos/esgotados/total` usando a derived query `countByEstoque(0)`.

Boas práticas – síntese

Controller

- Só HTTP: entrada, status e resposta
- `@Valid` nos DTOs de entrada
- 201 + Location, 204 na exclusão

Service

- Casos de uso e orquestração
- Injeção por construtor
- Exceções de domínio, sem HTTP

Model / Repository

- Regras no documento
- `BigDecimal` para dinheiro
- Derived queries antes de `@Query`

Transversais
DTOs no contrato da API · erros no padrão Problem Details · segredos em variáveis de ambiente · perfis por ambiente · testes em várias camadas · JAR executável.

Referências

- WALLS, Craig. **Spring Boot in Action**. Shelter Island: Manning, 2015.
- XESQUEVIXOS, Wanderson. **Learning Spring Boot 4**: simplify the development of production-grade applications using Java and Spring. 4. ed. Birmingham: Packt, 2026.
- SPRING. **Spring Boot Reference Documentation**. Disponível em: https://docs.spring.io/spring-boot/
- SPRING. **Spring Boot 4.0 Migration Guide**. Disponível em: https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-4.0-Migration-Guide
- SPRING. **Spring Data MongoDB – Reference Documentation**. Disponível em: https://docs.spring.io/spring-data/mongodb/reference/
- JAKARTA EE. **Jakarta Validation 3.1 Specification**. Disponível em: https://jakarta.ee/specifications/bean-validation/
- NOTTINGHAM, M.; WILDE, E.; DALAL, S. **RFC 9457: Problem Details for HTTP APIs**. IETF, 2023.

PUC Minas

Obrigado!

Prof. Rommel Vieira Carneiro